

HOW IT WORKS

Teaching a ledger to *read*

A bank statement says `MIGROS 6010 08.01.25`. A person says “groceries.” This is the story of the small machine that learned to close that gap — and how it ended up living inside enterprise software.

Written for people who have never trained a model. No mathematics beyond multiplication is required, and every number below was measured, not estimated.

CATEGORY ACCURACY 99.75% Which of ten spending categories a purchase belongs to	OCCASION F1 0.839 Whether it was everyday life, a date night, a trip or a gift
EXAMPLES LEARNED FROM 4,000 Rows of realistic Swiss card transactions	ANSWER TIME <1 ms No network call — the model runs inside the app itself

01 – The problem

Machines are bad at merchant names

Every card payment leaves behind a short, ugly string of text. Banks were never trying to make it readable — it is a routing label, not a description.

Here are four real examples from the training set, exactly as they appear:

RAW STATEMENT LINES	
MIGROS 6010 08.01.25	CHF 16.77
CAFÉ SPRÜNGLI PARADEPLATZ	CHF 12.50
SBB CFF FFS BILLET NR. 88213	CHF 41.00
RESTAURANT BLAUE ENTE	CHF 148.50

A human reads those instantly. A computer sees four unrelated blobs of characters. Worse, the same shop appears a dozen different ways — `MIGROS ZÜRICH HB`, `MIGROS MM 4021`, `Migros 6010 08.01.25` — so you cannot simply keep a list of shop names and look them up. There will always be a spelling you have not seen.

So instead of teaching the computer *names*, we teach it *shapes*. That turns out to be the whole trick.

02 – The dataset

Four thousand receipts that never happened

A model is mostly its data, so this deserves saying plainly: **the training data is synthetic**. Nobody's real bank statements were used. Every one of the 4,000 rows was generated by a script.

That was not a shortcut, it was the only honest option. Real card data for two people is both private and far too small — a couple produces a few hundred transactions a year, and the categories they never use produce none at all. So `ml/generate_data.py` builds a ledger that behaves like a real one: a fixed random seed, so it is reproducible to the byte, and roughly two years of spending between **15 June 2024** and **31 August 2026**.

ML/DATA/TRANSACTIONS.CSV · 4,000 ROWS, 7 COLUMNS

```
header                                date,time,merchant_raw,amount_chf,payer,category,moment

row 1                                2024-06-15,10:49,MIGROS 6010 08.01.25,16.77,A,Groceries,everyday
row 2                                2024-06-15,13:05,RESTAURANT SANTA LUCIA,52.56,A,Dining,everyday
```

Making it look like a real bank feed

A generator that emits a clean `MIGROS` every time would teach the model nothing, because reality never looks like that. So each merchant is drawn from a table of genuine Swiss businesses — Migros, Coop, Denner, SBB CFF FFS, Zeughauskeller, Sprüngli, Hallenstadion, Salt, Swisscom — and then deliberately mangled the way a card terminal would mangle it: random casing, a store number bolted on, a booking date or reference number embedded mid-string, city suffixes, and umlauts that sometimes survive and sometimes do not.

The classes are lopsided, on purpose

Real spending is not evenly distributed, and the generator does not pretend otherwise. This imbalance matters later, when we choose which score to trust.

CATEGORY	ROWS	OCCASION	ROWS
Dining	732	everyday	2,567
Groceries	715	trip	751
Transport	506	date night	396
Cafés	480	gift	286
Entertainment	357		
Travel · Gifts	276 each		
Subscriptions	245		
Home	229		
Health	184		

Where the occasion labels come from

Category is easy to generate: pick a merchant, you know its category. Occasion is the interesting one. Each merchant carries a quiet “romance prior” — the odds that an evening there was a date rather than a Tuesday — and

the generator combines that with the hour, the weekday and the amount through a logistic curve. Dinner at Kronenhalle at 20:30 on a Saturday for CHF 180 comes out `date_night` with high probability; the same restaurant at noon on a Wednesday for CHF 40 does not.

Then **8% of the occasion labels are flipped at random**. That is deliberate sabotage, and section 06 explains why a perfect score would have been the bad outcome.

THE HONEST CAVEAT

Synthetic data flatters a model. The generator and the classifier share an author, so the patterns the model learns are patterns somebody put there — real receipts are messier than any script imagines. The 99.75% below is a real measurement on data the model never saw during training, but it is a measurement on a rehearsal, not on opening night. The genuine test is the corrections loop: every time a person overrules the model in the running app, that real row is written down and folded into the next retrain. The model gets more honest the longer it is used.

03 — Words into numbers

A model cannot read. It can only add up.

This is the single most important idea on the page. A machine learning model is, underneath, an enormous addition sum. To use one on text, you must first convert the text into a long list of numbers. That conversion is called **featurisation**, and getting it right matters far more than the model itself.

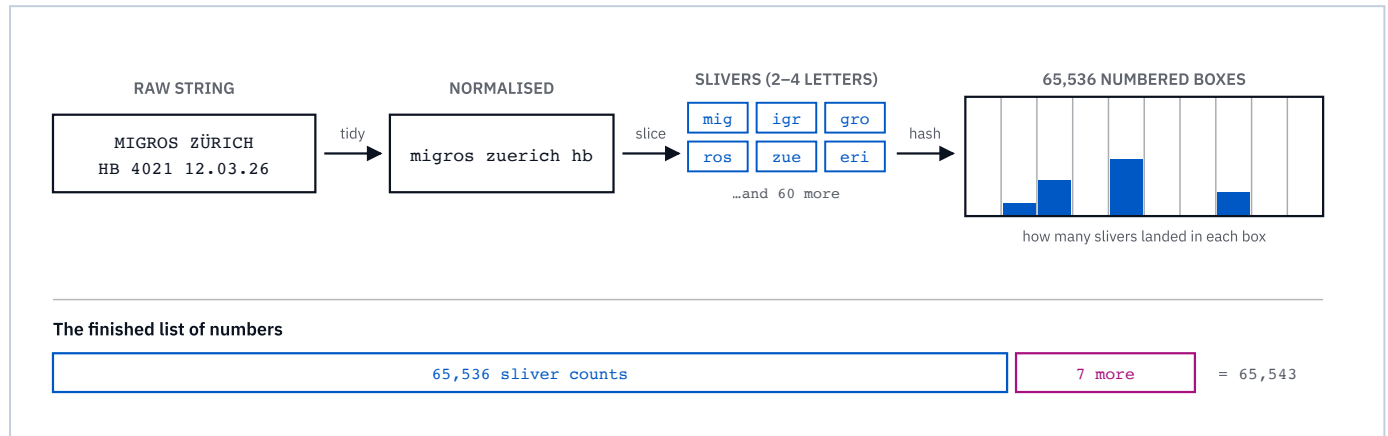
It happens in four steps. Follow one real merchant string all the way through:

WORKED EXAMPLE · ONE STRING, FOUR STEPS

0 · as it arrives	MIGROS ZÜRICH HB 4021 12.03.26
1 · tidy it up lowercase, ü→ue, drop the date and the store number	migros zuerich hb
2 · cut into slivers every 2-, 3- and 4-letter run, inside each word	" m", "mi", "ig", "gr" ...
3 · file each sliver 65,536 numbered boxes; count how many land in each	"mig" → box 41,207
4 · the result	a list of 65,543 numbers

Why slivers instead of whole words?

Because slivers survive mess. `MIGROS` and `Migros 6010` share almost every three-letter run — `mig`, `igr`, `gro`, `ros` — even though the full strings differ. A model that learns “`mig` + `igr` + `gro`” usually means “groceries” recognises spellings it has never seen. This is why the category accuracy is so high: Swiss shop names are extremely distinctive once you look at them in pieces.



One merchant string becomes a list of 65,543 numbers. Almost all of them are zero — only the boxes that actually received a sliver hold a count. Those seven extra numbers on the right are explained in the next section.

THE ONE CLEVER BIT

Why 65,536 boxes rather than a dictionary of every sliver ever seen? Because a dictionary grows forever and breaks the moment a new shop opens. Fixed boxes never grow. Two different slivers occasionally land in the same box — a collision — and the model simply learns to live with it. Trading a little precision for a permanently fixed size is a very old, very good engineering bargain.

04 – The seven extra numbers

What the words don't tell you

The merchant name says *what* was bought. It says nothing about *the occasion*. For that, the model gets seven more numbers describing the moment of the purchase.

log(amount)

The size of the bill, compressed. The gap between CHF 5 and CHF 50 matters far more than the gap between CHF 500 and CHF 545, and the logarithm captures exactly that.

is_weekend

1 on Saturday or Sunday, otherwise 0.

is_evening

1 after 18:00, otherwise 0.

hour_sin · hour_cos

The time of day as a position on a clock face. Two numbers, not one, so that 23:00 and 01:00 sit next to each other instead of at opposite ends of a scale.

dow_sin · dow_cos

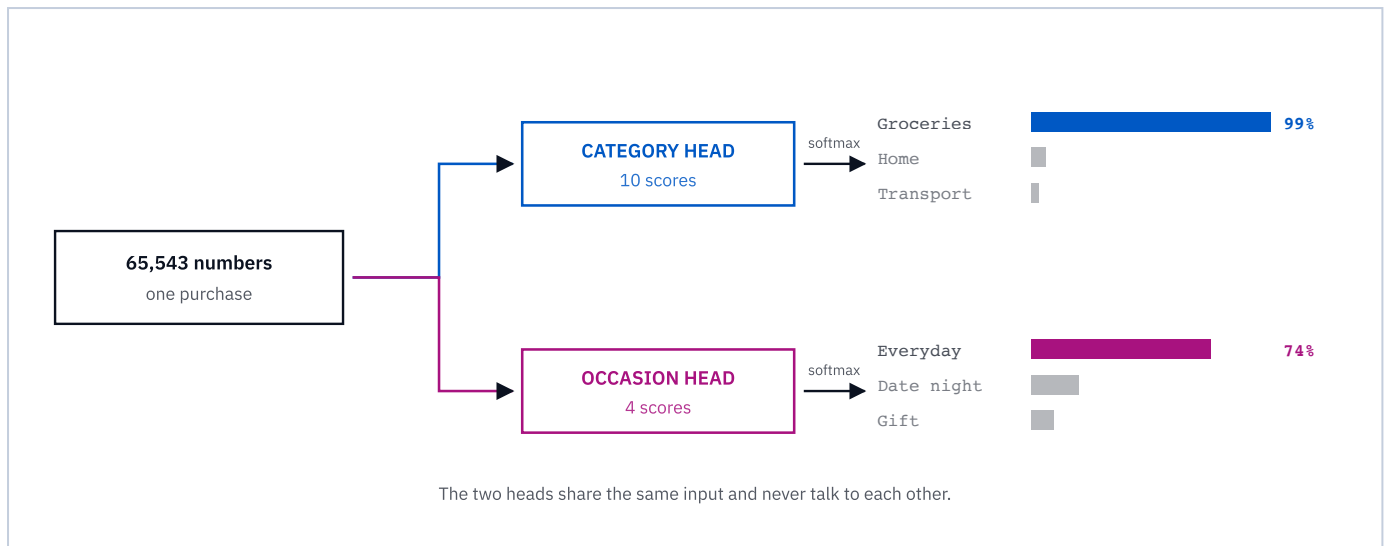
The same trick for the day of the week, so Sunday and Monday are neighbours.

That clock-face trick is worth pausing on, because it is a lovely piece of engineering taste. If you feed a model the hour as a plain number from 0 to 23, it concludes that 23:00 and 00:00 are twenty-three hours apart — the two most similar moments of the night, treated as maximally different. Placing the hour on a circle instead, and handing over its two coordinates, fixes the problem completely.

05 – Two questions, two heads

One model that answers twice

Every purchase raises two separate questions, and they are not the same question at all.



The real percentages for `MIGROS ZUERICH HB 4021 12.03.26`, CHF 43.15, on a Thursday evening: Groceries at 99.4%, Everyday at 74.0%.

What “softmax” means

Each head produces one raw score per option — ten scores for category, four for occasion. Those scores are unbounded and hard to interpret. Softmax is the step that turns them into percentages that add up to 100. That is all it is: a way of saying “99.4% sure it is groceries, 0.4% sure it is home supplies” instead of “7.2 versus 1.1”.

The percentage matters as much as the answer. Anything the model reports below **60% confidence** is not posted automatically — it is flagged for a human to confirm. A model that knows when it is unsure is far more useful than one that is merely often right.

06 – How it was trained

Nudging 65,543 numbers into shape

Training is less mysterious than it sounds. The model starts with every weight at zero, guesses, is told how wrong it was, and nudges each weight slightly in the direction that would have been less wrong. Then it repeats, up to two thousand times, until the nudges stop helping.

The model is **logistic regression** — one of the oldest and plainest tools in the box. No hidden layers, no neural network. Each of the 65,543 inputs gets one weight per possible answer; the score for “Groceries” is every input multiplied by its Groceries weight, all added together. That is the entire calculation. It trains in seconds and answers in under a millisecond.

The exact procedure

STEP	SETTING	WHY
Split	80 / 20, seed 42 stratified on category	Stratifying matters here: Health has only 184 rows, and an unlucky split could leave it nearly absent from the test set and make the score meaningless.
Scaling	StandardScaler	Fitted on the <i>training rows only</i> . Fitting it on everything first would leak information about the test set into training and inflate the result.
Solver	lbfgs, max_iter 2000	Converges reliably on a sparse problem this size.
Regularisation	C = 12.0	The interesting knob. The default of 1.0 held the weights so close to zero that the model was right but permanently unconfident — every answer around 60%. Since the app uses confidence to decide what to auto-post, that was useless. C = 12 lets the merchant n-grams speak.
Final refit	all 4,000 rows	Once the honest estimate is taken from the held-out fold, the shipped model is retrained on every row. Throwing away 20% of an already small ledger would cost real accuracy for no benefit.

That last row is a distinction worth keeping straight: **the numbers you are about to read were measured on 800 rows the model had never seen**, and the model that actually ships is a second one, trained the same way on all 4,000.

What came out — category

CATEGORY	PRECISION	RECALL	F1	HELD-OUT ROWS
Groceries	1.000	1.000	1.000	143
Dining	0.993	0.993	0.993	146
Cafés	0.990	1.000	0.995	96
Home	1.000	0.978	0.989	46
Transport · Travel · Gifts · Health · Entertainment · Subscriptions	1.000	1.000	1.000	369
Overall accuracy			0.998	800

Six of the ten categories are perfect. The only confusion is between Dining, Cafés and Home — which is exactly where a human hesitates too, since a café that serves lunch and a homeware shop with a restaurant are genuinely ambiguous.

What came out — occasion

OCCASION	PRECISION	RECALL	F1	HELD-OUT ROWS
everyday	0.903	0.934	0.918	518
trip	0.906	0.868	0.887	144
gift	0.898	0.772	0.830	57
date night	0.740	0.704	0.722	81
Macro average	0.862	0.820	0.839	800

Why the headline number is 0.839 and not 0.887

The occasion head gets **88.7% of individual rows right**. The number reported everywhere else in this article is **0.839**, and the lower one is deliberate.

Plain accuracy counts every row equally, so it is dominated by the biggest class — and `everyday` is 2,567 of the 4,000 rows. A lazy model that answered “everyday” to absolutely everything would still score 64%. **Macro F1** averages the score of each class separately, so the 286 gifts count exactly as much as the 2,567 everyday purchases, and there is nowhere for a lazy model to hide. Quoting the harder number is the point.

Date night is the weakest class at 0.722, and that is the correct outcome. It is the genuinely ambiguous label — a CHF 90 dinner on a Friday might be a date or might just be dinner — and it is the one carrying most of the deliberate 8% label noise.

WHY A PERFECT SCORE WOULD HAVE BEEN BAD NEWS

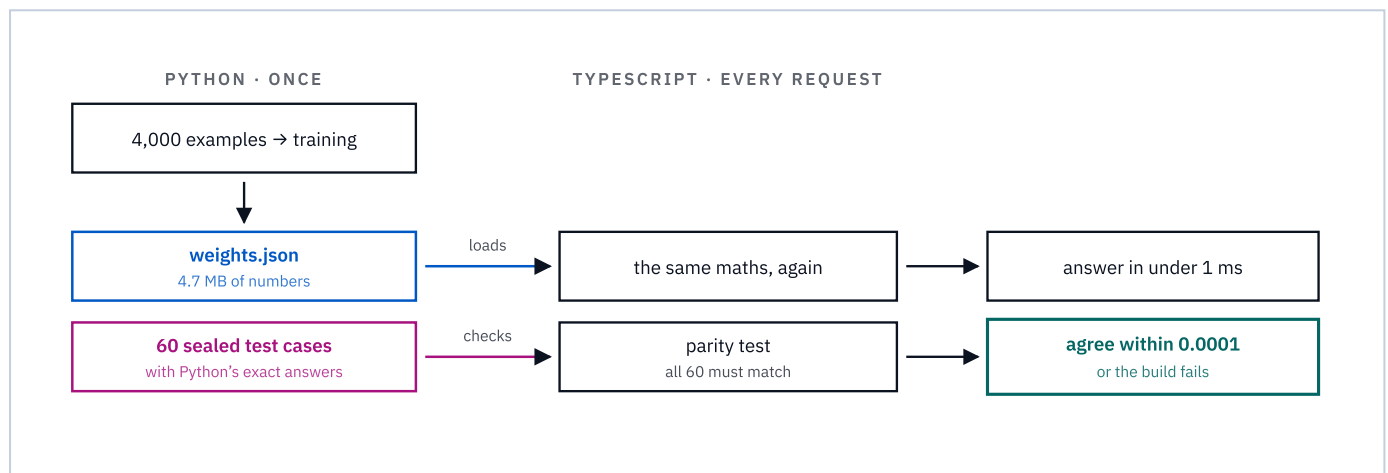
If the occasion head had scored 1.000, the correct response would be suspicion, not celebration. On a task where a thoughtful person could not always agree with themselves, a perfect score means the model memorised its examples rather than learning the pattern — and it would collapse on the first real receipt. The 8% of labels flipped at random in the generator make memorising impossible: the ceiling is deliberately below perfect, so a high score has to come from genuine signal. An honest 0.84 on a hard question beats a suspicious 1.00.

Trained in Python, working in TypeScript

Training happens in Python, because that is where the tools live. But the finished app is a TypeScript web application, and putting a Python process in the middle of every request would mean a slower app and a second thing to keep running.

So the model moves house. After training, its weights are exported to a single 4.7 MB file, and a second implementation — written in TypeScript — loads that file and does the arithmetic itself. Python trains; TypeScript answers.

Which raises the obvious danger: **two implementations can drift apart**. A rounding difference, a slightly different way of cutting slivers, and the app quietly starts giving different answers from the model that was actually tested. The fix is a bridge built specifically to catch that.



Before exporting, Python records its own answers for sixty randomly chosen purchases. The TypeScript version must reproduce all sixty to within one ten-thousandth. If it cannot, the tests fail and the code does not ship.

This is the part of the project I would defend hardest. It converts a vague worry — “are these two really the same model?” — into a question a machine answers on every single build. Ninety-one automated checks now guard that boundary.

THE SAME INPUT, BOTH LANGUAGES, MEASURED			
input	CAFÉ SPRÜNGLI PARADEPLATZ · CHF 12.50 · 09:10		
Python says	Cafes	0.997317	
TypeScript says	Cafes	0.997317	
difference	none, to six decimal places		

A small model inside big enterprise software

The model does not work alone. Getting from a photograph of a crumpled receipt to a posted, categorised, split expense takes three distinct pieces, and only one of them is the model.

PIECE	WHO BUILT IT	WHAT IT DOES
SAP Document AI	SAP	Reads the photograph. Pulls out the merchant, the date, the total and the line items.
The two-head classifier	Ours, from scratch	Decides the category and the occasion, and says how confident it is.
SAP CAP	SAP	The application framework the whole thing lives in — data model, services, the Fiori interface.

WORTH BEING PRECISE ABOUT

The classifier is **not** an SAP model, and it is not a large language model either. It is a purpose-built logistic regression trained specifically for this task, running inside an SAP application. SAP supplies the reading (Document AI) and the scaffolding (CAP); the categorising is bespoke. Conflating those three is the most common way this kind of write-up becomes untrue.

The loop that makes it better

Every time a person overrules the model — changing “Home” to “Gifts” — that correction is written down. When twenty corrections have accumulated, the model retrains overnight on the original examples plus the real ones, re-exports its weights, re-runs all sixty parity checks, and reloads itself without the app restarting. Being corrected is how it learns.

The fine-tuning question, answered honestly

The obvious next question is: why train your own model at all, when SAP has machine learning services? So we went and read the documentation — SAP's own 670-page service guide, the AI API specification, and the product guides — rather than recalling it. The answer is more interesting than expected.

THE HEADLINE

You cannot fine-tune SAP's models. Not in Document AI, and not in the generative AI hub. That is not a gap somebody forgot to fill — it is a deliberate position. SAP gives you pre-trained models to call, a governed runtime to call them in, and a place to run *your own* training. What it does not offer is a button that adjusts the weights of a model it hosts.

Layer by layer

LAYER	WHAT YOU REALLY CAN CUSTOMISE	WHAT YOU CANNOT
Document AI	Custom schemas (which fields to pull out), custom document types, per-field prompts, choice of LLM, and layout templates .	Train a model on your annotated documents. There are exactly three extraction methods — pre-trained models, generative AI, and templates. None of them produces a model of yours.
Generative AI hub	Prompt templates and a prompt registry, grounding / RAG , content filtering, data masking, and switching model with a config parameter.	Fine-tune a hosted model. There is no tuning endpoint and no LoRA capability anywhere in the API.
AI Core	Everything — this is the bring-your-own-model layer. Training runs as an Argo workflow from your own Docker image, on resource plans that go up to a V100 GPU, and the trained artefact is registered and served through the same API.	Nothing much. You supply the model, so you own the problem.
HANA PAL / APL	Genuine in-database training : classification, regression, gradient boosting, clustering and time series, trained and scored where the data already lives.	Anything resembling a language model. This is classical ML, and very good at it.

The sentence that nearly caught us out

SAP's own overview page for the generative AI hub says: *“Generative AI models are general by design, but you can fine-tune them with additional embeddings.”* Read quickly, that sounds like managed fine-tuning. It is not. It is describing **grounding** — retrieving relevant text and putting it in the prompt. No weights change. An article that quoted that line as evidence of fine-tuning would be repeating marketing prose as a technical claim, and any SAP architect would catch it immediately.

The plainest confirmation comes from SAP itself. Asked whether the weights of its own ABAP coding model can be obtained for further fine-tuning, SAP's FAQ answers simply: **no**.

CHECK THE SHELF LIFE BEFORE YOU BUILD ON IT

There *was* an SAP service that did almost exactly what our classifier does — **Data Attribute**

Recommendation, which trains on a CSV of text and categories and predicts the category of new rows. It is the obvious answer to this problem, and it is being wound down: SAP removed it from the list of eligible cloud services in **March 2026**, available only to the end of existing subscription terms. Business Entity Recognition went the same way a year earlier, and Service Ticket Intelligence before that. Had this project started six months earlier and reached for the obvious service, it would now be rebuilding. A 200-line model you own has a longer shelf life than a managed service you do not.

So what would we actually train next?

Knowing where the seams are, these are the realistic next steps for this app — each at the layer that genuinely supports it:

- **A receipt schema in Document AI.** Today it uses the standard invoice type. A custom schema with receipt-shaped fields — tip, VAT rate, table number, covers — would extract more per photograph. Schema work, not training.
- **Line-item categorisation.** Right now a whole receipt gets one category. A supermarket trip is really wine, nappies and flowers. The same two-head model, applied per line item, would split one posting into several — and needs no new technology at all.
- **Forecasting in HANA.** The spending forecast is Holt-Winters in TypeScript. PAL does time series in-database, so the same forecast could run where the data lives. This one is genuinely a fit for an SAP service.
- **Grounding the yearly statement.** Instead of passing a year of aggregates in the prompt, embed previous statements and memories and let the model retrieve them — so it can notice that you went back to the same restaurant three Februaries running. That is grounding, which the hub does support.
- **Retraining on AI Core.** The nightly retrain currently runs on the app's own machine. Packaged as a Docker image and an Argo workflow template, it would run on AI Core and serve through `CLASSIFIER_URL` — which the code already supports, precisely so this swap needs no code change.

Notice the shape of that list. Only one item is “use an SAP model”. The rest are either schema and prompt work, or our own model running on SAP's infrastructure. That is not a criticism of the platform — it is what the platform is actually for.

10 — Where it fails

The honest part

Any write-up that only lists successes is advertising. Here is a real failure, found by testing the finished model on inputs it had never seen.

A GENUINE MISS

input	NETFLIX.COM · CHF 24.90
correct answer	Subscriptions
model said	Groceries
confidence	0.40 — below the 0.60 threshold
what happened next	flagged for review, not posted

The cause is coverage, not mathematics. The training data contains `NETFLIX` but never `NETFLIX.COM`, and the extra slivers from `com` were enough to confuse it. The fix is more varied examples, not a cleverer model.

The consolation is that the system behaved correctly anyway. It was unsure, it knew it was unsure, and it asked rather than guessing. That is the behaviour worth engineering for — being right often, and being honest about the rest.

There is no neural network here, no cloud inference, no model with a name you would recognise. There is a list of 65,543 numbers, a very old statistical method, and a test that refuses to let two languages disagree.

It reads a receipt in under a millisecond, admits when it is lost, and gets a little better every time someone corrects it. For a ledger kept by two people, that is precisely enough.

VERIFIED · 91 PARITY CHECKS PASSING